

Workstations

Create New workstation

To automatically create a new workstation, run the script `createNewWorkstation` in the folder `workstations`.

To setup a new workstation manually, do the following:

1. Copy the folder of an existing workstation to eg. `newNameOfWorkstation`
2. Edit `tools/general.settings` with a new line eg. `workstation=newNameOfWorkstation;mac`.
The name of the workstat cannot contain spaces or special characters.
3. Edit `workstations/newNameOfWorkstation/settings.json`'s keys:

```
"os", "username", "password", "workstation"
```

See [settings below](#) for details.

After creation of the new workstation folder (automatically or manually), you can customize it.

1. Startup a local server (eg. through vscode)
2. Move things around, create windows, shortcuts, edit system icons etc
3. Use the `Export` function, copy the contents of the file to overwrite `workstations/newNameOfWorkstation/settings.json`
4. Edit this file manually to fit your needs; repeat steps 5-7 until you're happy.

Notes to each key

settings

Object with all general settings of this workstation. OS and workstation-name are defined in `general.settings` aswell!

- `"systemColor": "#000000"` HEX color code for background of taskbar, window headers etc. Font color is either white or black, depending on the darkness of this color
- `"loginGradient": "linear-gradient(...)"` Any valid background css property. If available, overwrites default gradient
- `"desktopColor": "#000000"` HEX color code for dekstop background
- `"desktopImg": "os/_generic/desktops/7.jpg"` Path to desktop image, either to `os/_generic/desktops/..` or `workstations/..../desktops/1.jpg`, see [below](#). Can also be `random` to select a different desktop image each time.
- `"os": "windows"` Style of OS, either `windows`, `mac` or `linux`. Must be the same as in `generalSettigns.txt`
- `"darkMode": true` only gets re-set by changing the `systemColor` via the UI
- `"username": "Some Name"` display name of workstation

- **"bootSpeed": 3** Integer speed of boot animation. 1 to 7, 1=fastest, 7=slowest. Default: 3.
- **"password": "1234"** Used for the logout action. Can be anything for the login window; to a successful login this only has to be part of the typed password
- **"workstation": "telefabi"** same name as folder in *workstations/..*
- **"selectedSystemMessage": 1** index of *osNotification* array; starts with 0
- **"osNotificationsDelay": 0** only gets re-set by changing the delay slider via the UI

systemIcons

Array of strings containig specific material-icon names. Add any icon name that you want to be displayed. If empty, no icons are shown.

The system time is an object with an array for hour & minutes.

All possible strings:

```
"expand_less", "account_circle", "wifi", "wifi_off", "usb", "bluetooth",
"gpp_maybe", "shield", "warning", "mail", "lan", "cloud", "volume_up",
"volume_mute", "watch_later", {"clock": ["12", "59"]} }
```

windows

Objects of live, displayed or hidden windows. accessible programs like terminal, browser or filemanager with a specific folder.

- **"windowName": "File manager"** display title in window - can be anything
- **"icon": "folder"** material-icon name, see [here](#)
- **"contentPath": "filemanager/index.html?folderContent=Downloads"** content of the window. Can be only name of folder in *programs/* or any specific html file with all the parameters you want
- **"x": 56** X-Position of window in percent (integer)
- **"y": 4** Y-Position of window in percent (integer)
- **"w": 598** Width of window in pixel (!) (integer)
- **"h": 444** Width of window in pixel (!) (integer)
- **"zIndex": 11** Starting at 10, this is the order which window overlapps which. The bigger this integer, the more in front it gets
- **"minimized": false** If true, window is minimized and added the taskbar or dock
- **"renderToDom": true** If false, this window is only rendered on demand. It does not show up on desktop OR the taskbar. It only is available through the *actionMenu* > "Saved and prepared windows". If you have a lot of windows that are not needed at the same time, setting this to false helps the performance massively

shortcuts

define desktop icons with dbl click action: 1: 'test.exe', 'folderFull.png', 250,650, ['action', 'path oder so']

- **"name": "DVD"** displayed filename (do not use " or ')
- **"icon": "dvd.png"** filename of icon image in *os/../../systemIcons/* - or, if empty AND **"action"** starts with 'addWindow', it shows the image in

programs/<programNameDefinedInAction>/icon.png. If "action" is also empty, it shows a default icon for that filename

- "x": 93 X-Position of shortcut in percent (integer)
- "y": 67 Y-Position of shortcut in percent (integer)
- "action": ".." javascript action as string like *startDefaultProgram('fileManager')* or *addWindow(..)*

actions

Arrays of strings with javascript in them. The get executed of selected in the actionMenu.

First key describes the title in the actionMenu; second key contains an arbitrary amount of javascript commands (including async/await actions).

osNotifications

Arrays of strings for different parts of the osNotification.

titel, message, icon, delay (if true==take delay from UI slider, else ms delay), duration, action onClick.

```
{
  "metaTitle": "Scene #1 For your eyes only",
  "title": "Zombie wave imminent",
  "description": "Hack into these brains to gain access to valuable
recources (..)",
  "icon": "warning",
  "initialDelay": true,
  "timeOut": 0,
  "action": "startDefaultProgram('terminal')"
```

- **metaTitle** Title in the actionMenu
- **title** Notification title in popup-box
- **description** Notification text
- **icon** material-icon name, see [here](#)
- **initialDelay** Delay: Boolean *true* or integer. If *true*: take delay from the UI slider in the actionMenu. If integer >=0: Delay time until the message shows up in **ms**. *0* also overwrites the UI slider.
- **timeOut** Duration of how long the message is shown. If *0*, it stays open indefinitely.
- **action** Optional js action that happens onclick of the osNotification. Anyways the message closes *onLick*

User and desktop images

Place the **user image** named *userpicture.jpg* into *workstations/./userpicture.jpg*. It is displayed when user ist "logged out".

If none is defined, this image falls back to *os/_generic/userpicture.jpg*.

Place any **desktop images** into `workstations/../../desktops/...jpg`. The filenames have to be consecutive integers, starting with **1**.

Files (`.jpg` or `.mp4`) in `workstations/../../files/...jpg` can be displayed by the `imageView` program.

The filenames can be anything.

To view these files in the `imageView`, the actions of a shortcut may look like this:

```
addWindow('Image viewer', 'image', 'imageviewer/index.html?
files=1.jpg|3.jpg|1.mp4|2.mp4|2.jpg', x, y, 666,450, false);
```

To open the images in the `imageView` by clicking on a file in the `fileManger`, add to any image "file" in the `folders.json` this to the last element of the array:

```
["IMG_20211210_173805.jpg", "", "filesize",
"1.jpg|3.jpg|1.mp4|2.mp4|2.jpg"]
```

See [below](#) for details about this array.

Browser settings

The settings of the browser are set in `workstations/../../browser.json`.

The websites are accessible globally (eg, for all workstations) and are stored in `programs/browser/sites/...`

fileManager folders

The *main* file & folder list displayed in the filemanager are set in `workstations/../../folders.json`.

An object marks a folder; an array marks a file.

The first object key has to be called **Root**.

```
{
  "Root": [
    {
      "Desktop": [
        ["My little pony script.py", "action", "filesize", "data"],
        ["A-433", "action", "filesize", "data"],
        {"Subfolder": []},
      ],
      "Other folder 1": [],
      "Other folder 2": [],
      ...
    }
  ]
}
```

A file is an array and is defined as follows:

```
["filename.jpg", "action", "filesize", "data"],
```

The icon of the file is determined by the filename.

filesize is not yet used anywhere.

If **action** is empty (or **"action"**), the system tries to start a default application (eg. for an image the `imageView`, etc).

If **data** is empty (or **"data"**), depending on the filetype, the default action tries to do something with that data. Eg, if the filename is `My Text.txt` and the last key data is `This is the content of the text editor that opens`, the `textEditor` that opens on `dblclick` contains the aforementioned text.

Fun fact: In this case, if the data key is **"random"**, the text editor will display one of three different paragraphs from Goethe, Kafka, or some other text about copyright.